

Assignment - 2

A Lab Report submitted by

Palak Mishra

B24EE1050

OBJECTIVE

The objective of this assignment is to implement and verify a 32-bit single-bus datapath using Verilog HDL.

The datapath consists of 32-bit registers, a common 32-bit bus, a multiplexer, and a 32-bit ALU. The ALU supports ADD, SUB, AND, and XOR operations. The Y and Z registers are used as intermediate registers during ALU operations.

The following micro-operations were implemented and verified:

1. $R2 \leftarrow R1$
2. $R3 \leftarrow R1 + R2$
3. $R3 \leftarrow R1 - R2$
4. $PC \leftarrow PC + 4$
5. DATAPATH ARCHITECTURE

The implemented datapath follows the single-bus architecture given in the assignment.

The major components of the datapath are:

- 32-bit Program Counter (PC)
- 32-bit Memory Address Register (MAR)
- 32-bit Memory Data Register (MDR)
- 32-bit Instruction Register (IR)
- Four 32-bit general-purpose registers (R0–R3)
- 32-bit Y register
- 32-bit Z register
- 32-bit common bus
- 2-to-1 multiplexer
- 32-bit ALU
- Carry-in and carry-out signals
- Register enable and bus-selection control signals

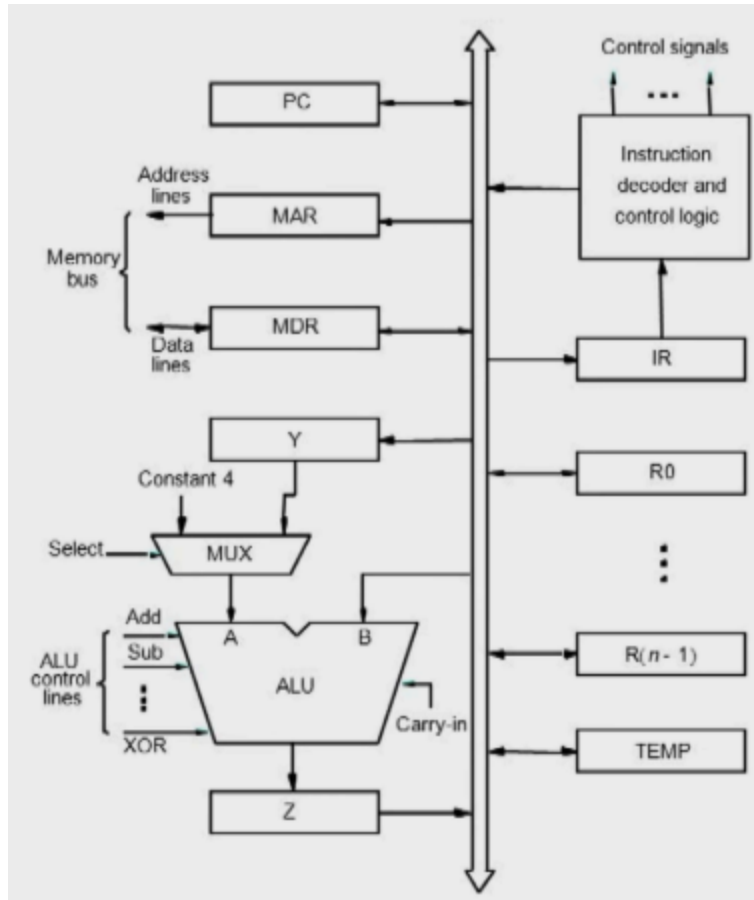


Figure 1: Single-Bus Datapath Architecture

REGISTER IMPLEMENTATION

A reusable 32-bit register module was implemented in Verilog. Each register has a clock, reset, enable, 32-bit data input, and 32-bit data output.

The register is updated on the rising edge of the clock when its enable signal is asserted. When reset is active, the register is cleared to zero.

The same register module is instantiated for all the required registers:

PC, MAR, MDR, IR, R0, R1, R2, R3, Y, and Z.

This modular approach avoids duplicating the register logic and ensures consistent operation across the datapath.

COMMON BUS

A 32-bit common bus is used for transferring data between the registers. The source placed on the bus is selected using the bus_select control signal.

The implemented bus sources are:

Bus Select Source

0 PC

1 MAR

2 MDR

3 IR

4 R0

5 R1

6 R2

7 R3

8 Y

9 Z

10 External Data

Only one source is selected onto the bus at a time. A destination register loads the value from the bus when its corresponding enable signal is asserted.

For example, the operation $R2 \leftarrow R1$ is implemented as: $R1 \rightarrow \text{BUS} \rightarrow R2$

Thus, the data transfer takes place through the common bus rather than by directly connecting R1 to R2.

MUX AND ALU

The ALU has two inputs. The B input of the ALU is connected directly to the common bus. The A input is obtained from a 2-to-1 multiplexer. The MUX selects between the Y register and the constant value 4.

MUX Selection ALU A Input

0 Y

1 Constant 4

The ALU supports four operations: 00 ADD, 01 SUB, 10 AND, 11 XOR

The result produced by the ALU is stored in the Z register.

Therefore, the general ALU data flow is:

First Operand $\rightarrow Y \rightarrow \text{MUX} \rightarrow \text{ALU}$

Second Operand $\rightarrow \text{Common Bus} \rightarrow \text{ALU}$

ALU Result $\rightarrow Z$

The Z register can then place the result back onto the common bus for transfer to the required destination register.

MICRO-OPERATION 1: $R2 \leftarrow R1$

The first required operation transfers the contents of R1 to R2.

The data flow is: $R1 \rightarrow \text{BUS} \rightarrow R2$

The following sequence is used:

1. R1 is selected as the source of the common bus.
2. The r2_enable signal is asserted.
3. At the active clock edge, R2 loads the value present on the bus.

For the simulation:

R1 = 10

After the operation:

R2 = 10

The simulation confirms that the contents of R1 are successfully transferred to R2 through the common bus.

```
=====
INITIALIZING R1 = 10
=====
```

```
=====
INITIALIZING R2 = 20
=====
```

```
=====
INITIALIZING PC = 100
=====
```

```
=====
TEST 1 : R2 <- R1
=====
```

```
-----
PC  = 100
MAR = 0
MDR = 0
IR  = 0
R0  = 0
.
```

```
-----
R1  = 10
R2  = 10
R3  = 0
Y   = 0
Z   = 0
BUS = 10
-----
```

MICRO-OPERATION 2: $R3 \leftarrow R1 + R2$

Since the architecture has a single common bus, both operands cannot be transferred to the ALU simultaneously. Therefore, the Y register is used to temporarily store the first operand.

Cycle 1: $R1 \rightarrow \text{BUS} \rightarrow Y$

Therefore: $Y = 10$

Cycle 2: $R2 \rightarrow \text{BUS}$

At the same time, Y is selected through the MUX and supplied to the A input of the ALU.

Therefore:

ALU A = 10

ALU B = 20

The ALU performs:

$10 + 20 = 30$

The result is stored in Z: $Z = 30$

Cycle 3: $Z \rightarrow \text{BUS} \rightarrow R3$

Therefore: $R3 = 30$

Simulation result: $R1 = 10$, $R2 = 20$, $Y = 10$, $Z = 30$, $R3 = 30$

Hence, the operation $R3 \leftarrow R1 + R2$ was successfully verified.

```
=====
TEST 2 : R3 <- R1 + R2
=====
```

```
-----
PC   = 100
MAR  = 0
MDR  = 0
IR   = 0
R0   = 0
R1   = 10
R2   = 20
R3   = 30
Y    = 10
```

```
-----
Z    = 30
BUS  = 30
-----
```

MICRO-OPERATION 3: $R3 \leftarrow R1 - R2$

The subtraction operation follows the same general sequence as the addition operation.

Cycle 1: $R1 \rightarrow \text{BUS} \rightarrow Y$

Therefore: $Y = 10$

Cycle 2: $R2 \rightarrow \text{BUS}$

The MUX selects Y, and the ALU performs: $Y - \text{BUS}$

Therefore: $10 - 20 = -10$

The result is stored in Z.

Since the datapath is 32-bit, -10 is represented using two's-complement representation. When displayed as an unsigned decimal number, this value appears as: 4294967286

Cycle 3: $Z \rightarrow \text{BUS} \rightarrow R3$

Therefore: $R3 = 4294967286$

This corresponds to -10 when interpreted as a signed 32-bit value. Hence, the subtraction operation was successfully verified.

```
=====
TEST 3 : R3 <- R1 - R2
=====
-----
PC   = 100
MAR  = 0
MDR  = 0
IR   = 0
R0   = 0
R1   = 10
R2   = 20
R3   = 4294967286
Y    = 10
Z    = 4294967286
BUS  = 4294967286
-----
```

MICRO-OPERATION 4: $PC \leftarrow PC + 4$

This operation uses the constant-4 input of the MUX.

For the simulation: $PC = 100$

Cycle 1: $PC \rightarrow BUS \rightarrow Y$

Therefore: $Y = 100$

Cycle 2: The MUX selects the constant 4. The PC value is placed on the common bus and supplied to the B input of the ALU.

Therefore:

ALU A = 4

ALU B = 100

The ALU performs: $4 + 100 = 104$

The result is stored in Z: $Z = 104$

Cycle 3: $Z \rightarrow BUS \rightarrow PC$

Therefore: $PC = 104$

Hence, the operation $PC \leftarrow PC + 4$ was successfully verified.

```
=====
TEST 4 : PC <- PC + 4
=====
-----
PC   = 104
MAR  = 0
MDR  = 0
IR   = 0
R0   = 0
R1   = 10
R2   = 20
R3   = 4294967286
Y    = 100
Z    = 104
BUS  = 104
-----
```

ADDITIONAL ALU VERIFICATION

In addition to the four required micro-operations, the AND and XOR operations were also tested to verify the remaining ALU functionality.

AND operation:

R1 = 10

R2 = 20

10 AND 20 = 0

The simulation produced: Z = 0

XOR operation:

10 XOR 20 = 30

The simulation produced: Z = 30

Thus, all four ALU operations—ADD, SUB, AND, and XOR—were verified successfully.

EXTRA TEST : R1 AND R2

=====

PC = 104
MAR = 0
MDR = 0
IR = 0
R0 = 0
R1 = 10
R2 = 20
R3 = 4294967286
Y = 10
Z = 0
BUS = 20

EXTRA TEST : R1 XOR R2

=====

PC = 104
MAR = 0
MDR = 0
IR = 0
R0 = 0
R1 = 10
R2 = 20
R3 = 4294967286
Y = 10
Z = 30
BUS = 20

=====

SIMULATION COMPLETE

=====

SIMULATION RESULTS

The following table summarizes the simulation results.

Micro-operation Input Values Result

$R2 \leftarrow R1$ $R1 = 10$ $R2 = 10$

$R3 \leftarrow R1 + R2$ $R1 = 10$, $R2 = 20$ $R3 = 30$

$R3 \leftarrow R1 - R2$ $R1 = 10$, $R2 = 20$ $R3 = -10$

$PC \leftarrow PC + 4$ $PC = 100$ $PC = 104$

$R1 \text{ AND } R2$ $10 \text{ AND } 20$ $Z = 0$

$R1 \text{ XOR } R2$ $10 \text{ XOR } 20$ $Z = 30$

Note: The subtraction result is displayed as 4294967286 when interpreted as an unsigned 32-bit value. This is the two's-complement representation of -10.

WAVEFORM VERIFICATION

The complete simulation waveform was generated using the Verilog testbench. The waveform shows the clock, reset, bus-selection signals, register enable signals, MUX selection, ALU control signals, and datapath values. The waveform verifies the sequential movement of data through the common bus and the intermediate Y and Z registers.

For register-transfer operations, the data follows:

Source Register $\rightarrow$ Common Bus $\rightarrow$ Destination Register.

For ALU operations, the data follows:

Source Register $\rightarrow$ Common Bus $\rightarrow$ Y $\rightarrow$ MUX $\rightarrow$ ALU $\rightarrow$ Z $\rightarrow$ Common Bus $\rightarrow$ Destination Register

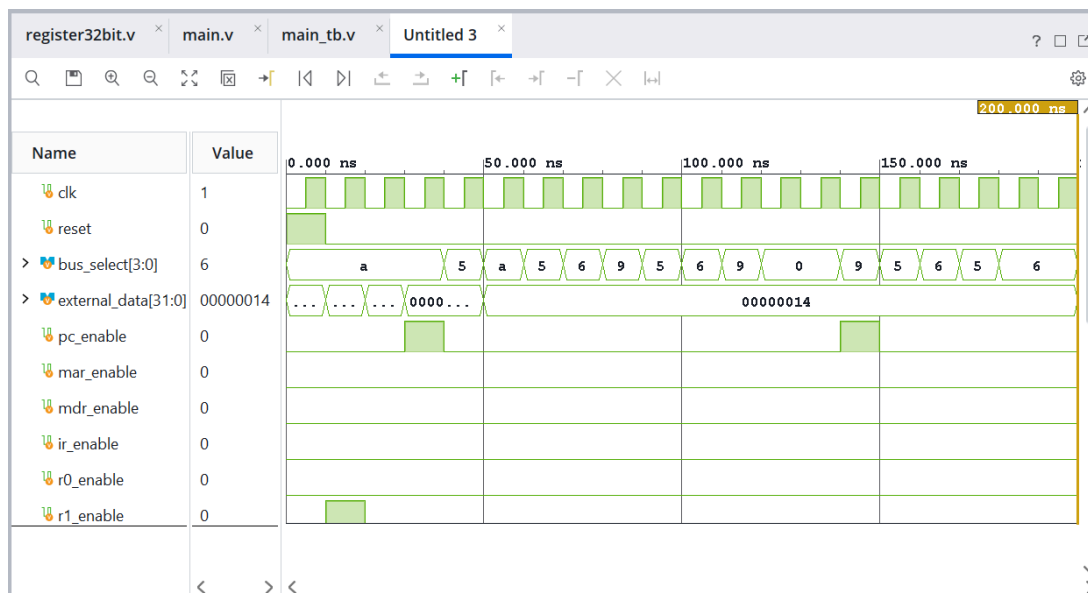


Figure 2: Simulation Waveform

SOURCE CODE AND SIMULATION FILES

The complete Verilog source code, testbench, and simulation waveform have been uploaded to the GitHub repository.

GitHub Repository: https://github.com/palakmishra01/COA_LAB/tree/main/Ass02

The repository contains

- register32bit.v
- main.v
- main_tb.v
- Simulation waveform

CONCLUSION

The 32-bit single-bus datapath was successfully implemented and verified using Verilog HDL.

The implementation includes the required registers, common bus, MUX, and ALU. The ALU supports ADD, SUB, AND, and XOR operations. The Y and Z registers are used as intermediate registers during ALU operations.

The four required micro-operations : $R2 \leftarrow R1$, $R3 \leftarrow R1 + R2$, $R3 \leftarrow R1 - R2$, and $PC \leftarrow PC + 4$; were successfully demonstrated through simulation.

The simulation results and waveform confirm that the required data transfers and ALU operations are performed through the common bus according to the specified single-bus datapath architecture.